

Multi-Level Feedback Queue Scheduler

Week 3: Boosting and Polishing

Aadesh Panjwani and M. Rayyan Khan

Contents

1	Overview	3
2	Priority Boosting Implementation	3
2.1	Global Boost Interval	3
2.2	Boost Procedure	3
2.3	Testing Syscall	3
3	Scheduler Integration	4
4	Demo and Testing	4
4.1	mlfqdemo User Program	4
4.2	Sample Output	5
4.3	Observations	5
5	Conclusion	5

1 Overview

This report presents the completion of the Multi-Level Feedback Queue (MLFQ) scheduler in xv6-RISC-V, focusing on **starvation prevention** and final polishing.

Key goals accomplished:

- Implement periodic **priority boosting** to prevent starvation.
- Expose `boostproc()` syscall for deterministic testing.
- Validate MLFQ behavior with both CPU-bound and I/O-bound processes using `mlfqdemo`.

2 Priority Boosting Implementation

2.1 Global Boost Interval

- Introduced a global constant:

```
1 #define MLFQ_BOOST_INTERVAL 200
```

- Every timer tick calls `scheduler_tick()`, which invokes `mlfq_maybe_boost()` on CPU 0 to avoid multiple boosts.

2.2 Boost Procedure

- Clears all per-level queues.
- Resets every non-UNUSED process to **level 0** with a fresh time slice.
- Re-queues any RUNNABLE process at the highest priority.
- Running tasks inherit the new level next time they yield or block.

```
1 void mlfq_force_boost(void)
2 {
3     last_boost_tick = ticks;
4     mlfq_boost_all(); // resets queue_level & time_slice_budget
5 }
```

2.3 Testing Syscall

- Exposed via `boostproc()` syscall. - Allows **deterministic manual boost** from user space.

```
1 uint64 sys_boostproc(void)
2 {
3     mlfq_force_boost();
4     return 0;
5 }
```

3 Scheduler Integration

Timer interrupts only trigger `yield()` if **time slice is exhausted**, preserving fairness for I/O-bound tasks.

`scheduler_tick()` now:

- Charges CPU ticks to the current process.
- Decrements time slice and handles demotion if slice is consumed.
- Calls `mlfq_maybe_boost()` to periodically enforce priority boosting.

4 Demo and Testing

4.1 mlfqdemo User Program

- Spawns a **CPU-bound** worker and an **I/O-bound** worker.
- Periodically samples and prints each process:

- Current queue level
- Remaining time slice
- Total runtime

- Supports optional argument to invoke `boostproc()` at a specific iteration.

```
1 // simplified example from mlfqdemo.c
2 if(boost_iter >= 0 && i == boost_iter){
3     printf("mlfqdemo: invoking boostproc() at iteration %d\n", i);
4     boostproc();
5 }
```

4.2 Sample Output

```
$ mlfqdemo
iteration 0:
  cpu(pid=4): level=1 budget=7 total=5
  io (pid=5): level=0 budget=4 total=0
iteration 1:
  cpu(pid=4): level=2 budget=16 total=12
  io (pid=5): level=0 budget=4 total=0
iteration 2:
  cpu(pid=4): level=2 budget=10 total=18
  io (pid=5): level=0 budget=4 total=0
iteration 3:
  cpu(pid=4): level=2 budget=4 total=24
  io (pid=5): level=0 budget=4 total=0
iteration 4:
  cpu(pid=4): level=3 budget=31 total=29
  io (pid=5): level=0 budget=4 total=0
iteration 5:
  cpu(pid=4): level=3 budget=25 total=35
  io (pid=5): level=0 budget=4 total=0
mlfqdemo: invoking boostproc() at iteration 5
iteration 6:
  cpu(pid=4): level=1 budget=7 total=42
  io (pid=5): level=0 budget=4 total=0
iteration 7:
  cpu(pid=4): level=2 budget=15 total=50
  io (pid=5): level=0 budget=4 total=0
iteration 8:
  cpu(pid=4): level=2 budget=9 total=56
  io (pid=5): level=0 budget=4 total=0
iteration 9:
  cpu(pid=4): level=2 budget=2 total=63
  io (pid=5): level=0 budget=4 total=0
$ █
```

Figure 1: CPU-bound process gradually demotes to lower-priority queues until priority boost is applied. I/O-bound process remains in high-priority queue due to frequent yielding.

4.3 Observations

- **CPU-bound process** drifts downward through levels $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$. Since the level-0 time quantum is low, priority is dropped before a log is generated, thus we only see level-1.
- **I/O-bound process** remains at top priority (level 0), ensuring low latency.
- After a manual or automatic boost, all processes are **reset to level 0**.
- Scheduler demonstrates expected fairness and prevents starvation for CPU-heavy tasks.

5 Conclusion

- Final MLFQ scheduler supports **multiple queues, per-process time slices, and priority boosting**.
- CPU-bound processes are demoted appropriately and periodically boosted to prevent starvation.
- I/O-bound processes remain responsive, demonstrating interactive-friendly scheduling.
- `mlfqdemo` confirms the correctness of the scheduler across different workloads.